Didar Customer Retention Modeling Challenge

Comprehensive Machine Learning and MLOps Technical Report

Objective: prioritize exactly 50 customers whose churn risk is expected to decrease the most as a result of proactive contact.

Amirhossein Izadi

AI/ML Engineer

July 2026

Document Control

Project	Didar Customer Retention Modeling Challenge
Author	Amirhossein Izadi
Report type	Comprehensive technical report and release-readiness review
Primary evidence	Challenge brief; `report.md`; source-code snapshot; JSON/CSV metrics and ranking artifacts
Data version	`didar_retention_v1`
Feature version	`2.2.0` in model outputs; dataset manifest retains `2.1.0` and requires synchronization
Model version	`1.1.0`
Reported champion	`logistic_t`: L2-regularized logistic T-learner
Theme	Didar brand color `#049B8C`

Evidence and interpretation convention

The report distinguishes four evidence levels. **Artifact-verified** values are read directly from machine-readable JSON/CSV outputs. **Code-verified** statements are supported by the supplied source. **Project-reported** statements appear in the supplied report but were not independently rerun in a fully locked environment. **Recommended** items are forward-looking engineering controls rather than claims of completed implementation.

Executive Summary

The challenge asks for a reproducible method that combines customer attributes with 60 days of product-usage history and produces a ranked list of exactly 50 customers for a retention campaign. The project correctly reframes this as a **causal uplift / conditional average treatment effect problem**: the goal is not to find the customers most likely to churn, but those whose churn probability is expected to fall most when contacted.

The supplied implementation is substantial: the archive contains **341 files**, including **104 Python source files** with approximately **5661 lines** under `src/`, a typed CLI, data contracts, four causal meta-learner families, Optuna tuning, MLflow tracking, DVC stages, FastAPI serving, Docker Compose, Prometheus/Grafana configurations, and **102 test functions** distributed across unit, integration, leakage, contract, robustness, property, and golden-test categories.

0.8036	0.1626	0.5692	0.3578
holdout factual ROC-AUC	holdout Brier score	reported DR policy value @50	mean predicted uplift @50

The reported champion is the **logistic T-learner**. It uses two independently regularized logistic outcome models, one trained on historically uncontacted customers and one on contacted customers. This comparatively simple architecture outperforms the evaluated CatBoost S/T/DR/X variants on the reported holdout policy metric. That result is plausible for a 4,000-customer dataset because each treatment arm contains only about 2,000 observations, the feature representation is heavily engineered, and high-capacity boosted models can add variance to the difference between two potential-outcome predictions.

The project demonstrates strong predictive signal: a conventional churn-risk logistic baseline reaches approximately 0.801 ROC-AUC, while the final factual holdout AUC is 0.8036. More importantly, the causal ranking differs from risk ranking. The reported doubly robust estimate for the risk-based top-50 policy is -0.1097, whereas the uplift champion is reported at +0.5692. This comparison supports the project thesis that high churn risk is not equivalent to high contact responsiveness.

Important interpretation boundary

The value 0.5692 is a model-based observational policy estimate, not an experimentally observed 56.9 percentage-point retention lift. Historical contact was confounded by account-manager targeting, and conditional ignorability cannot be verified. The final ranking should therefore be validated through a randomized or partially randomized retention campaign before its estimated causal magnitude is treated as operational truth.

The strongest aspects of the project are the causal framing, chronological feature design, actual development/holdout split, cross-fitting, broad test surface, explicit policy layer, and production-oriented packaging. Before an external or production release, several artifacts should be synchronized: model metadata alternates between logistic and CatBoost champion labels; the stored bootstrap interval describes the overall mean DR pseudo-outcome rather than the top-50 policy value; the final ranking contains zero uncertainty for every selected customer; and the stability block in the final summary is empty. These issues do not invalidate the full engineering effort, but they are important release gates for a senior-quality submission.

Key findings

- Historical contact was targeted, not random.
- Logistic T-learner is the reported champion.
- Holdout AUC is 0.8036 and Brier is 0.1626.
- Risk-only targeting is estimated below random.
- All inspected holdout propensities remain in support.

Immediate release actions

- Synchronize champion and feature metadata.
- Recompute a true top-50 policy interval.
- Persist repeat-level predictions and rank stability.
- Replace zero per-customer uncertainty.
- Validate impact with a randomized pilot.

Contents

Document Control i

Executive Summary i

1 Challenge Definition and Success Criteria 1
1.1 Original task . . . 1
1.2 Why churn prediction is insufficient . . . 1
1.3 Acceptance criteria . . . 1

2 Data Assets and Exploratory Analysis 2
2.1 Dataset profile . . . 2
2.2 Treatment and outcome balance . . . 2
2.3 Evidence of targeted treatment assignment . . . 3
2.4 Usage behavior . . . 3
2.5 Segment risk . . . 4
2.6 Feature redundancy . . . 4

3 Feature Engineering Design 5
3.1 Feature inventory . . . 5
3.2 Chronological correctness . . . 6
3.3 Missingness semantics . . . 6
3.4 Ablation requirement . . . 6

4 Validation, Cross-Fitting, and Leakage Control 6
4.1 Development and holdout split . . . 6
4.2 Repeated cross-fitting . . . 6
4.3 Leakage register . . . 7

5 Causal Learner Architectures 7
5.1 S-learner . . . 7
5.2 T-learner . . . 7
5.3 DR-learner . . . 8
5.4 X-learner . . . 8

6 Hyperparameter Optimization and Model Selection 8
6.1 Optuna strategy . . . 8
6.2 Selection criterion . . . 8
6.3 AutoML and TabFM status . . . 8

7 Predictive and Policy Results 8
7.1 Full model comparison . . . 9
7.2 Why logistic regression wins here . . . 9
7.3 Champion metrics . . . 10
7.4 Policy baselines . . . 11

8 Calibration, Support, and Subgroup Diagnostics 11
8.1 Calibration . . . 11
8.2 Propensity overlap . . . 12
8.3 Subgroup performance . . . 13

9 Ranking Output and Business Interpretation 13
9.1 Top-ranked customers . . . 13
9.2 Ranking policy . . . 15
9.3 Boundary sensitivity . . . 15

10 Explainability 15
10.1 Global drivers . . . 15
10.2 Interpretation requirements . . . 16

11 Reproducibility, MLOps, and Deployment Architecture 16
11.1 Repository and environment . . . 16

11.2 End-to-end architecture . . . 17
11.3 Reproduction commands . . . 17
11.4 MLflow and DVC . . . 17
11.5 Model bundle . . . 17

12 Serving, Input Contracts, and Failure Engineering 17
12.1 FastAPI surface . . . 17
12.2 Input policy . . . 18
12.3 Readiness behavior . . . 18

13 Monitoring and Operations 18
13.1 Three-dashboard model . . . 18
13.2 Feature drift . . . 18
13.3 Prometheus design . . . 19
13.4 Airflow . . . 19

14 Testing and Quality Assurance 19

15 Causal Limitations and Experimental Validation 20
15.1 Unmeasured confounding . . . 20
15.2 Treatment definition . . . 20
15.3 Recommended experiment . . . 21

16 Release-Readiness Review 21
16.1 Readiness scorecard . . . 22

17 Recommendations and Next Steps 22
17.1 Immediate submission actions . . . 22
17.2 Modeling roadmap . . . 22
17.3 Production roadmap . . . 23

18 Conclusion 23

A One-Page Challenge Submission Summary 24

B Complete Top-50 Ranking 24

C Feature-Drift Detail 27

D Source and Artifact Inventory 27

List of Figures

1 Potential-outcome formulation used for customer prioritization. 1
2 High-level dataset profile. 2
3 Observed treatment-outcome composition. 3
4 Selected standardized differences between contacted and uncontacted customers. . 3
5 Aggregate snapshots of median usage intensity across progressively recent windows. 4
6 Observed churn rates across subscription and package segments. 4
7 Selected pairwise feature correlations reported by the project. 5
8 Validation architecture used for model development and final evaluation. 6
9 Meta-learner families evaluated by the project. 7
10 Holdout DR policy value across causal model variants. 9
11 Holdout discrimination and probability accuracy. 10
12 Reported holdout policy values for the champion and operational baselines. 11
13 Reliability curves for the two treatment-arm predictions. 12
14 Summary of holdout propensity support. 12
15 Predicted potential outcomes for the top ten ranked customers. 13
16 Revenue, estimated uplift, and propensity among the selected cohort. 14
17 Decision-policy flow used for batch ranking. 15
18 Selected global contribution magnitudes across potential outcomes and uplift explanation. 16
19 Production-oriented ML system architecture. 17
20 Highest reported PSI values between development and holdout features. 19
21 Test-function inventory by category in the supplied snapshot. 20

1 Challenge Definition and Success Criteria

1.1 Original task

The supplied interview task describes a CRM company that records customer activity and historical retention contacts. The next campaign can contact exactly 50 customers. The required solution must use both customer-level data and the complete 60-day product-usage history, explore and explain the data, develop a prioritization approach, and produce a ranked list of 50 customer identifiers. The brief explicitly states that reasoning, evaluation, and interpretation matter more than model complexity.

Business decision

For each eligible customer, estimate the expected reduction in churn probability caused by proactive contact, then allocate the fixed capacity of 50 contacts to the highest-benefit customers.

1.2 Why churn prediction is insufficient

A churn model estimates $P(Y = 1 \mid X)$ and ranks customers by risk. That is useful for forecasting but does not answer whether contact changes the outcome. A customer can have high churn risk and zero treatment responsiveness; another can have moderate baseline risk and a large recoverable component. The required target is therefore:

$$\tau(x) = P(Y = 1 \mid T = 0, X = x) - P(Y = 1 \mid T = 1, X = x).$$

Here $T = 1$ denotes proactive contact and $Y = 1$ denotes churn. Positive $\tau(x)$ means the model expects contact to reduce churn.

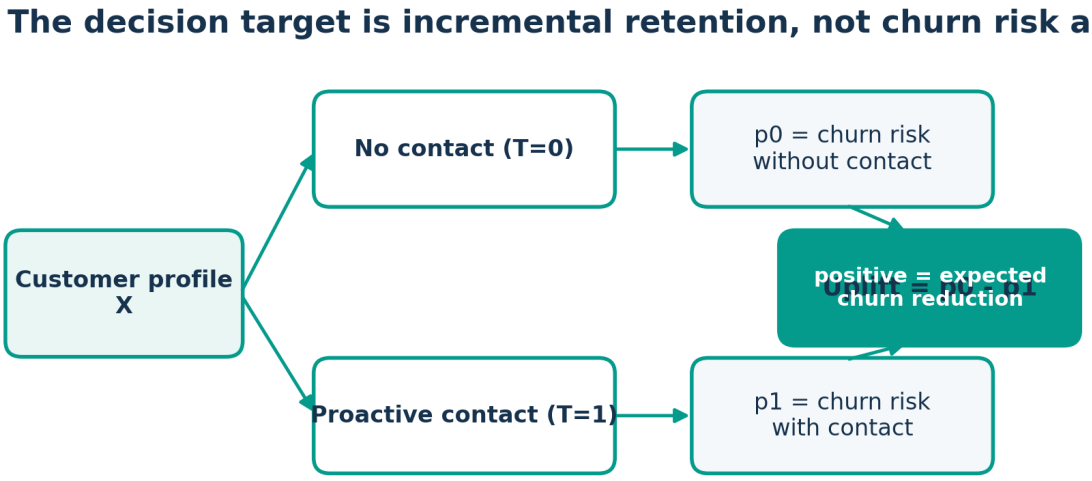


Figure 1: Potential-outcome formulation used for customer prioritization.
Source: Project methodology and causal learner implementation.

1.3 Acceptance criteria

A defensible solution should satisfy five classes of criteria:

1. **Decision correctness**: exactly 50 unique eligible customers, deterministic ranking, explicit score interpretation.
2. **Statistical validity**: holdout isolation, cross-fitting, factual outcome metrics, propensity support checks, causal assumptions documented.
3. **Model quality**: calibrated potential outcomes, positive incremental policy value, stable ranking, plausible effects.
4. **Engineering quality**: reproducible environment, data lineage, one deployable bundle, tested API, controlled exceptions.

5. **Governance**: version consistency, model card, audit trail, limitations, randomized validation plan.

2 Data Assets and Exploratory Analysis

2.1 Dataset profile

The project uses two sources: `customer_data.csv` with static customer attributes, historical treatment, and outcome; and `usage_timeseries.csv` with 60 daily usage observations per customer. The supplied quality artifact reports complete coverage and no missing static fields.

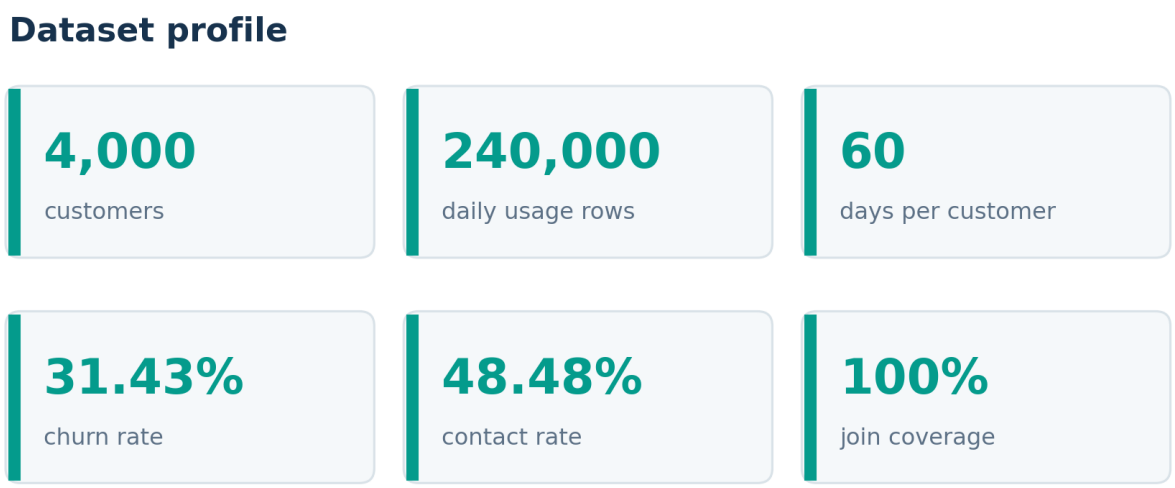


Figure 2: High-level dataset profile.
Source: metrics/data_quality.json.

Table 1: Data-integrity checks from the supplied quality artifact

Check	Result
Customer records	4,000
Usage records	240,000
Usage days per customer	60 minimum, median, and maximum
Customers with full history	4,000
Customers with partial history	0
Customers without usage	0
Usage records without customer	0
Join coverage	100%
Static missing values	0 across reported fields

2.2 Treatment and outcome balance

The population is reasonably balanced for treatment, with 1,939 contacted and 2,061 uncontacted customers. Churn affects 1,257 customers. The smallest treatment-outcome cell contains more than 600 customers, so severe class imbalance is not the primary modeling problem.

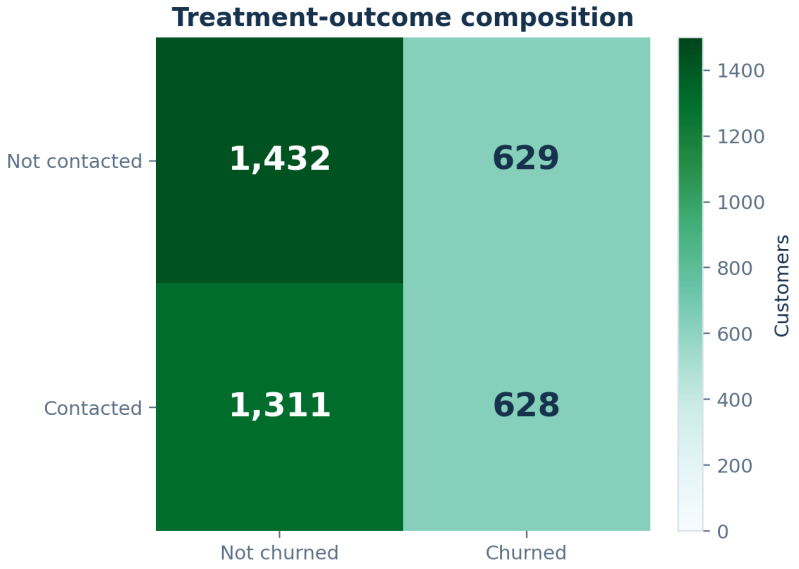


Figure 3: Observed treatment-outcome composition.
Source: Project report and data-quality artifact.

The naive observed churn rate is 32.39% among contacted customers and 30.52% among uncontacted customers. This does not imply that contact increases churn. The contacted cohort was intentionally selected based on risk signals, creating **confounding by indication**.

2.3 Evidence of targeted treatment assignment

A cross-validated propensity model reaches a reported ROC-AUC of 0.649, which means historical contact can be predicted from pre-treatment features better than chance. The largest standardized differences show that account managers preferentially contacted customers with deteriorating usage and more support activity.

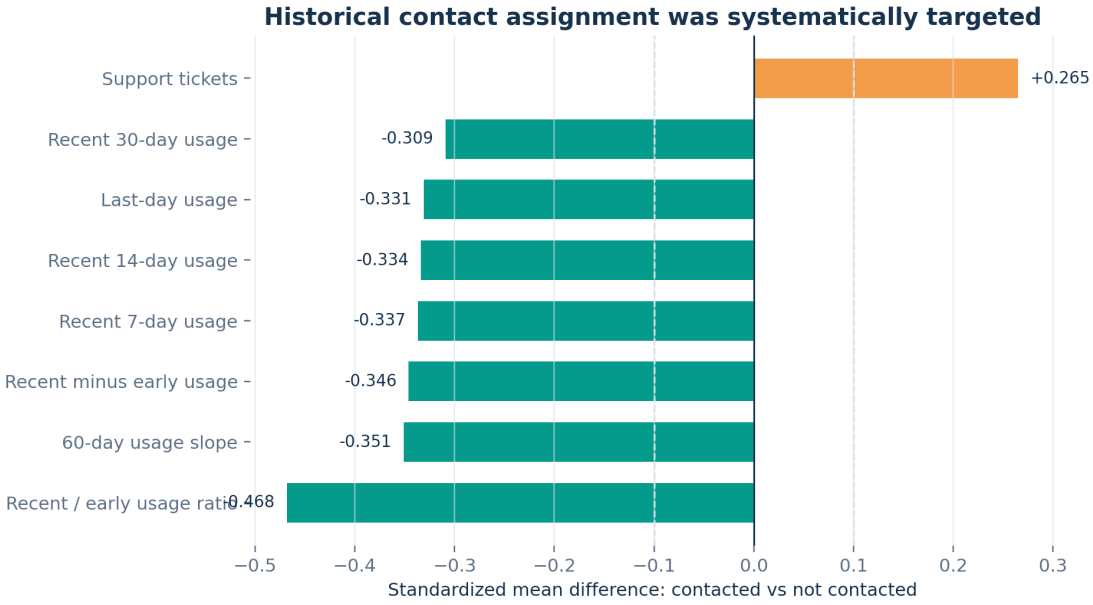


Figure 4: Selected standardized differences between contacted and uncontacted customers.
Source: Values reported in `report.md`. Negative values indicate lower measurements among contacted customers.

2.4 Usage behavior

The usage history exhibits a broad decline toward the decision date. The median early-30-day activity is 10.17 actions per day, compared with 3.17 over the recent 30 days and 0.86 over the recent 7 days. The median recent-to-early ratio is 0.395 and the median 60-day slope is -0.189 actions per day. Because decline is common across the population, the severity and shape of decline are more informative than the sign of the slope alone.

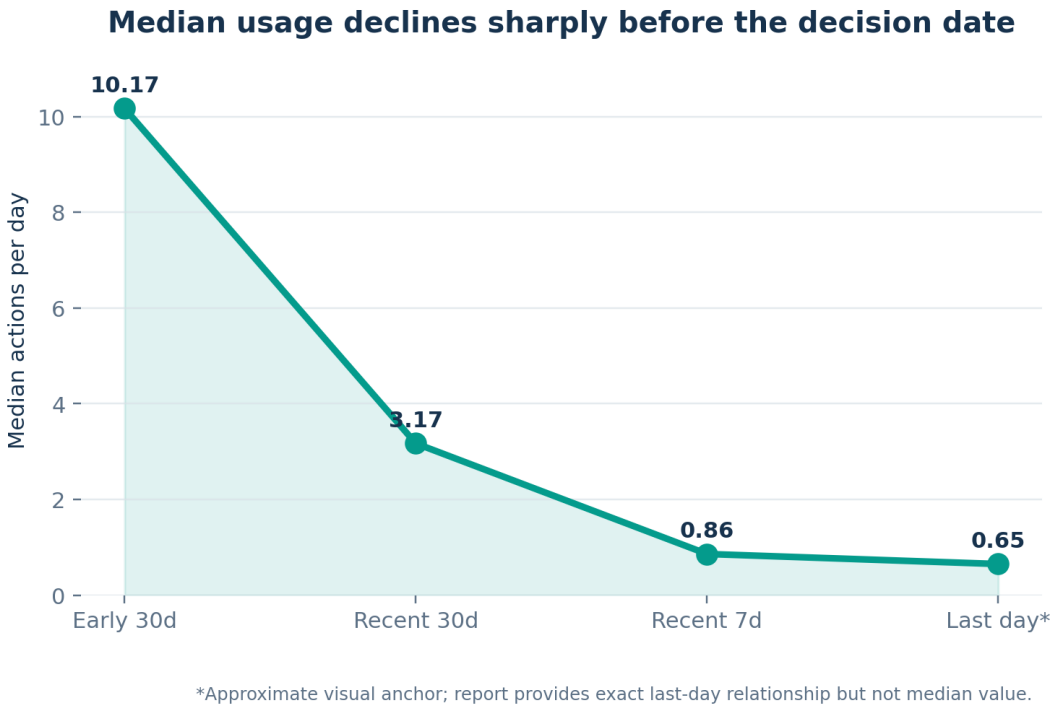


Figure 5: Aggregate snapshots of median usage intensity across progressively recent windows.
Source: Project report; the final plotted point is included only as a visual anchor and is not used as a claimed exact median statistic.

2.5 Segment risk

Monthly subscriptions have a materially higher observed churn rate than annual subscriptions. Package-level differences are smaller but still useful for predictive modeling and subgroup diagnostics.

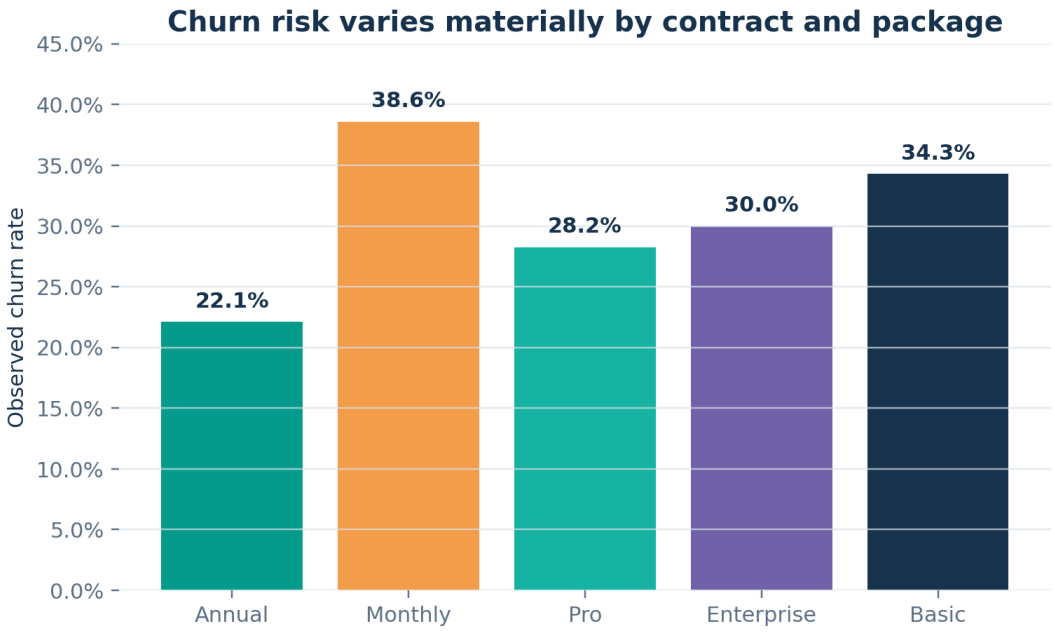


Figure 6: Observed churn rates across subscription and package segments.
Source: Project report.

2.6 Feature redundancy

Many usage aggregates describe the same latent behavior. Recent usage, zero-day counts, usage mean, and recent-to-early ratios are strongly correlated. Correlation does not make these features unusable, but it affects model selection and explanation. Regularized linear models can distribute weight smoothly, whereas tree ensembles may split repeatedly on near-duplicate signals and generate unstable local counterfactual differences.

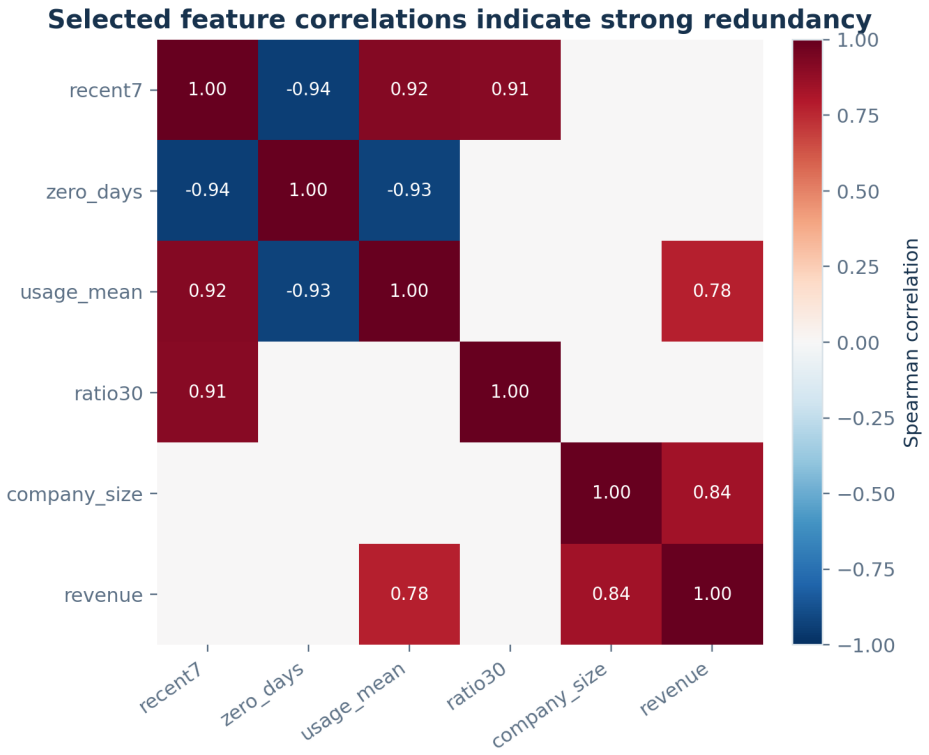


Figure 7: Selected pairwise feature correlations reported by the project.
Source: Spearman correlations from `report.md`. Blank cells were not supplied and are not imputed.

3 Feature Engineering Design

3.1 Feature inventory

The final manifest contains 53 features:

- 10 static and ratio features;
- 33 temporal aggregates;
- 5 PCA components from the 60-day usage sequence;
- 5 one-hot encoded category indicators.

Table 2: Feature families

Family	Count	Representative variables
Static	10	log company size, log revenue, revenue per employee, tenure in years, annual/monthly flags, support-ticket ratios, engagement
Level and recency	12+	usage mean/median/total, recent 7/14/30-day means, early-window means, last-day usage
Trend	6+	14/30/60-day slopes, recent minus early, recent-to-early ratio, percentage decline
Inactivity	6	zero days, missing days, active-days percentage, longest zero streak, days since activity, recent zero days
Volatility and shape	9+	standard deviation, coefficient of variation, daily change, down-day proportion, drawdown, residual variance, half-window totals and ratio
Sequence compression	5	PCA usage components 0 through 4
Categorical	5	package and subscription one-hot variables

3.2 Chronological correctness

The implementation explicitly orders daily usage columns from day -59 through day 0. This is important because lexicographic ordering would place names such as day -1 before day -10 and silently corrupt slopes, recent windows, and PCA inputs. The repository includes a robustness test for column-order invariance.

3.3 Missingness semantics

The production-facing design distinguishes missing observations from observed zero usage. This is a strong operational decision: a telemetry failure must not be interpreted as customer inactivity. Although the supplied challenge data is complete, the feature set includes missing-day indicators and the API policies support incomplete histories.

3.4 Ablation requirement

The feature architecture is broad, but the archive does not include a complete machine-readable ablation result in the supplied snapshot. A final scientific report should compare compact engineered features, the full 53-feature set, raw lags, and PCA. In particular, PCA captures sequence variance rather than treatment relevance and should remain only if it improves policy value and stability.

4 Validation, Cross-Fitting, and Leakage Control

4.1 Development and holdout split

The project creates a stratified 80/20 split: 3,200 development customers and 800 final holdout customers. Stratification uses the joint distribution of treatment and outcome, preserving the four critical cells. Unlike an earlier prototype design, the current training runner reads the split data through the dedicated loading path and reports development and holdout metrics separately.

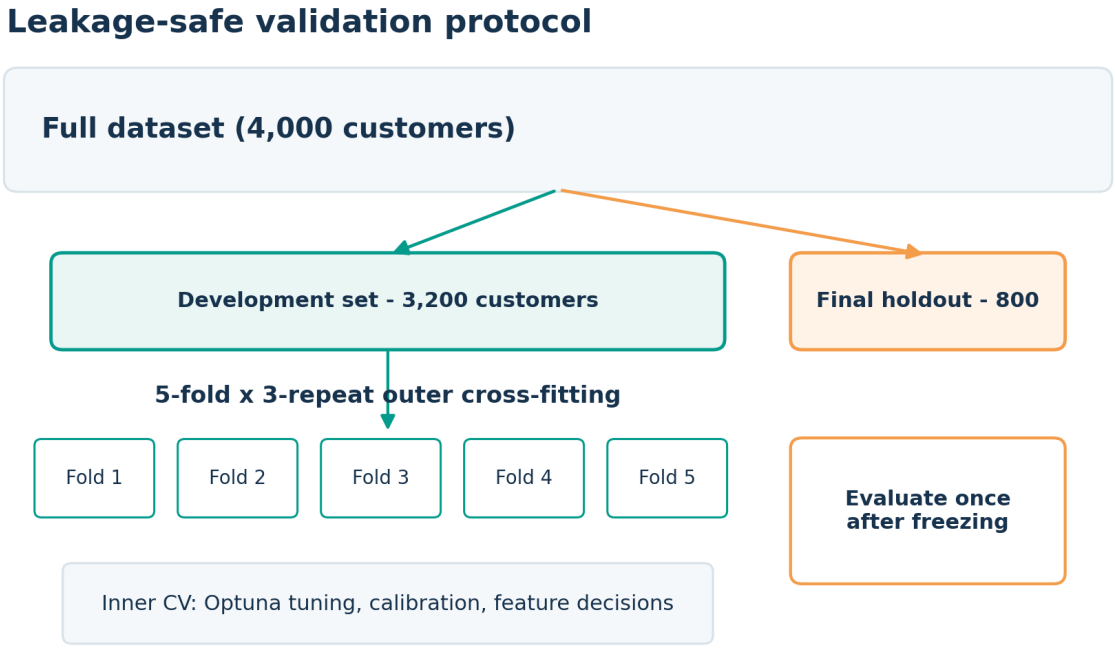


Figure 8: Validation architecture used for model development and final evaluation.
Source: `configs/validation.yaml`, split manifests, and training modules.

4.2 Repeated cross-fitting

Development predictions are generated through five folds and three repeats. In each fold, the preprocessor is fitted only on the training subset; the validation subset is transformed with that fitted object and receives out-of-fold predictions. For T-, DR-, and X-learners, the propensity and treatment-arm outcome models are also fitted within the fold.

The principal benefit is not only a less biased metric. Cross-fitting supplies the nuisance predictions

required by doubly robust estimation and prevents a row from evaluating a model that has already learned its outcome.

4.3 Leakage register

The source includes an explicit leakage register covering label leakage, treatment misuse, preprocessing, calibration, oversampling, tuning, interaction screening, blending, stacking, ranking, model selection, and reporting. This is a strong governance practice. The highest-priority controls are:

- no future usage or post-contact data;
- preprocessing, PCA, calibration, and resampling inside training folds;
- Optuna restricted to inner CV;
- model selection based on development OOF metrics;
- final holdout inspected after the architecture and policy are frozen.

5 Causal Learner Architectures

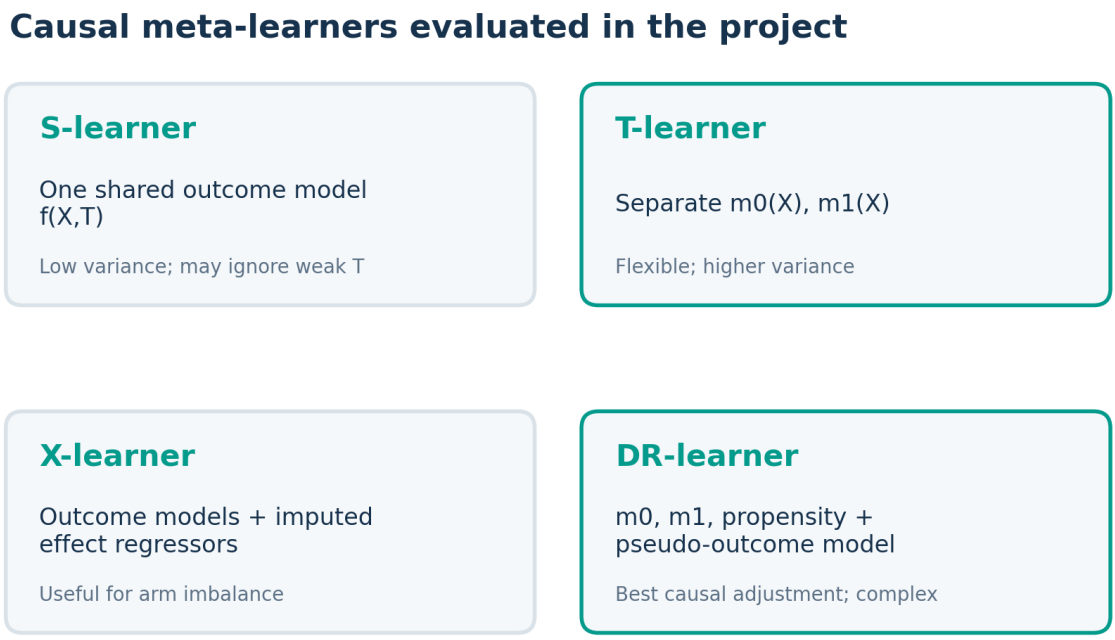


Figure 9: Meta-learner families evaluated by the project.
Source: Source modules under src/retention/modeling/train/.

5.1 S-learner

The S-learner fits one outcome model $f(X, T)$ using all observations. Potential outcomes are generated by scoring every customer twice, once with $T = 0$ and once with $T = 1$. This design shares baseline churn structure and is statistically efficient, but a flexible model may underuse a weak binary treatment feature when ordinary churn predictors dominate.

The project evaluates logistic, interaction-augmented logistic, and CatBoost S-learners. The holdout DR policy values are reported as 0.1389 for logistic S and 0.3232 for CatBoost S.

5.2 T-learner

The T-learner fits separate outcome functions $m_0(X)$ and $m_1(X)$. It permits distinct response surfaces, but each model receives only one treatment arm. The difference $m_0 - m_1$ can amplify calibration and variance errors. The project evaluates logistic and CatBoost variants, with the logistic

version selected as champion.

5.3 DR-learner

The DR learner first generates cross-fitted m_0, m_1, and propensity $e(X)$ predictions, then constructs:

$$\psi_i = m_0(X_i) - m_1(X_i) + \frac{(1-T_i)(Y_i - m_0(X_i))}{1-e(X_i)} - \frac{T_i(Y_i - m_1(X_i))}{e(X_i)}.$$

An Elastic Net model is fitted to the pseudo-outcomes. The project tunes this CATE stage with Optuna. This architecture is appropriate for observational treatment, but the pseudo-outcomes can be noisy and depend on overlap and nuisance-model quality.

5.4 X-learner

The X-learner imputes treatment effects separately within each treatment arm and combines two effect models using propensity. It is particularly useful when treatment-arm sizes are unequal. Here the arms are close to balanced, so the X-learner is a secondary challenger rather than the natural default.

6 Hyperparameter Optimization and Model Selection

6.1 Optuna strategy

The project uses Optuna's TPE sampler and fixed seeds. Logistic regularization and CatBoost depth, learning rate, L2 penalty, and minimum leaf support are tuned for S- and T-learners. Elastic Net and Ridge effect stages are tuned within DR and X training runs.

Representative selected parameters include:

- logistic T-learner: $C \approx 0.0255$;
- logistic S-learner: $C \approx 0.0313$;
- CatBoost T-learner: depth 3, learning rate approximately 0.026, L2 approximately 25.7;
- CatBoost S-learner: depth 4, learning rate approximately 0.011, L2 approximately 34.8.

The shallow and strongly regularized tree settings are suitable for the limited treatment-arm sample size.

6.2 Selection criterion

The training runner filters models using a development Brier threshold below 0.20 and then selects the highest development DR policy value at capacity 50. This is better aligned with the business decision than selecting solely by AUC. However, any top-k policy metric is noisy at $k = 50$; repeated-split stability and confidence intervals should participate in the final release gate.

6.3 AutoML and TabFM status

The project configuration includes optional dependency groups for AutoGluon and TabFM, but the final model-comparison artifact does not contain AutoGluon or TabFM results. They should therefore be described as research-ready challengers, not completed benchmark results. AutoML should remain nested inside the outer training fold, and TabFM requires explicit licensing, context-order, batch-composition, latency, and memory assessment.

7 Predictive and Policy Results

7.1 Full model comparison

Table 3: Model comparison from the supplied machine-readable artifact

Model	Dev AUC	Dev Brier	Dev DR@50	Test AUC	Test Brier	Test DR@50	Uplift@50
logistic_s_int	0.8124	0.1592	-0.0558	0.7894	0.1673	0.1389	0.1124
logistic_s	0.8124	0.1592	-0.0558	0.7894	0.1673	0.1389	0.1124
catboost_s	0.8080	0.1609	0.2983	0.7879	0.1677	0.3232	0.0933
logistic_t	0.8165	0.1576	0.3826	0.8036	0.1626	0.5692	0.3578
catboost_t	0.8081	0.1612	0.3202	0.7991	0.1649	0.4312	0.3283
logistic_dr	0.8151	0.1581	0.3776	0.8036	0.1626	0.3527	0.2551
catboost_dr	0.8072	0.1615	0.3453	0.7981	0.1654	0.3955	0.2582
logistic_x	0.8151	0.1581	0.3776	0.8036	0.1626	0.3889	0.3017
catboost_x	0.8072	0.1615	0.3453	0.7981	0.1654	0.3455	0.2905
blend_t	–	–	–	0.8036	0.1627	0.5557	–

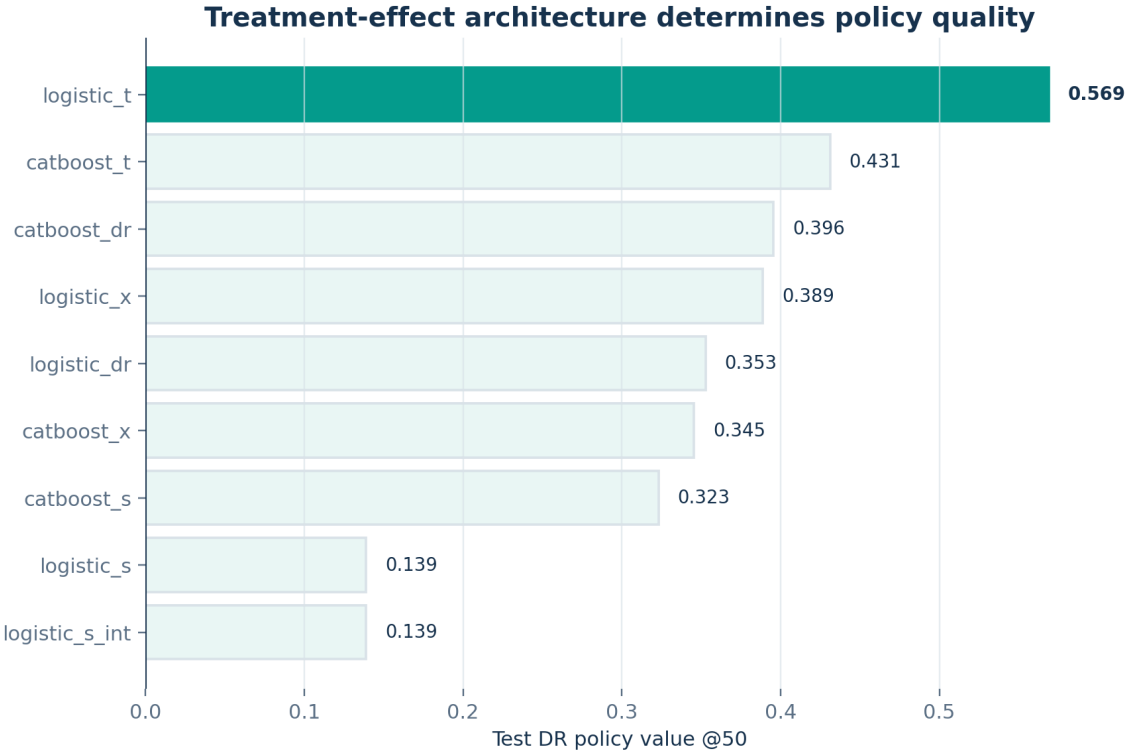


Figure 10: Holdout DR policy value across causal model variants.
Source: reports/model_comparison.csv.

7.2 Why logistic regression wins here

The result is technically reasonable. Each T-learner arm has roughly 2,000 observations and the input already contains engineered nonlinear summaries. Logistic regression offers low variance, smooth potential-outcome surfaces, strong regularization, and comparatively stable probability estimates. Boosted trees can discover additional interactions, but when two independently learned surfaces are subtracted, local probability noise directly becomes uplift noise.

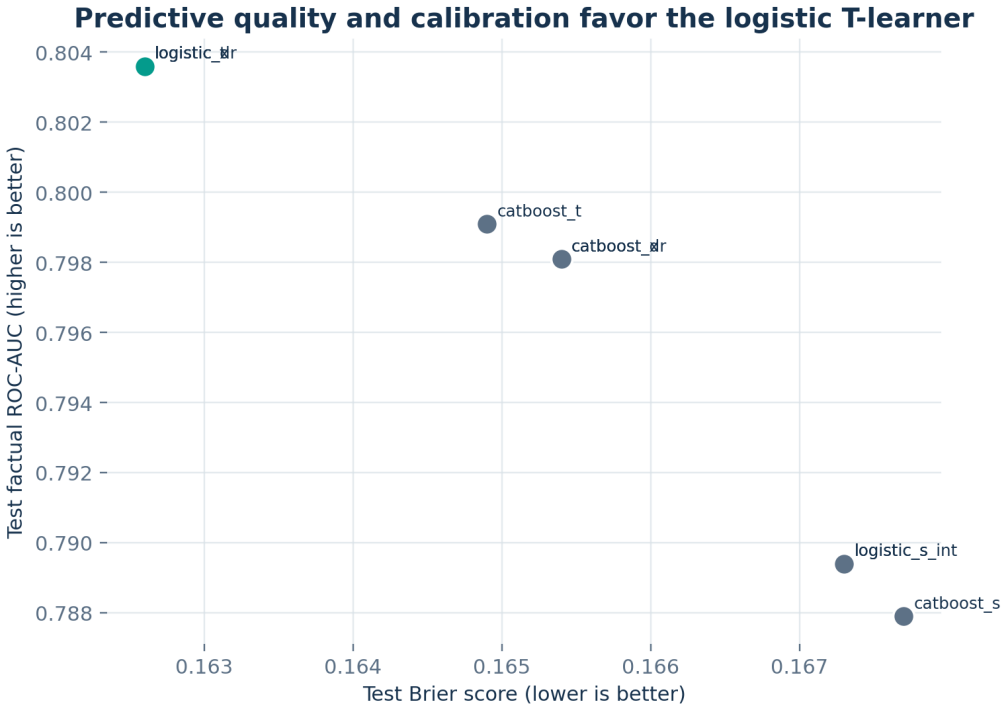


Figure 11: Holdout discrimination and probability accuracy.
Source: reports/model_comparison.csv.

7.3 Champion metrics

Table 4: Reported champion performance

Metric	Holdout value
Factual ROC-AUC	0.8036
Factual PR-AUC	0.6394
Factual log loss	0.4885
Factual Brier score	0.1626
Control-arm ROC-AUC	0.7800
Treated-arm ROC-AUC	0.8309
Control-arm Brier	0.1709
Treated-arm Brier	0.1539
AUUC	27.5710
Qini	35.1378
DR policy value @50	0.5692
Mean predicted uplift	0.0930
Mean predicted uplift @50	0.3578

7.4 Policy baselines

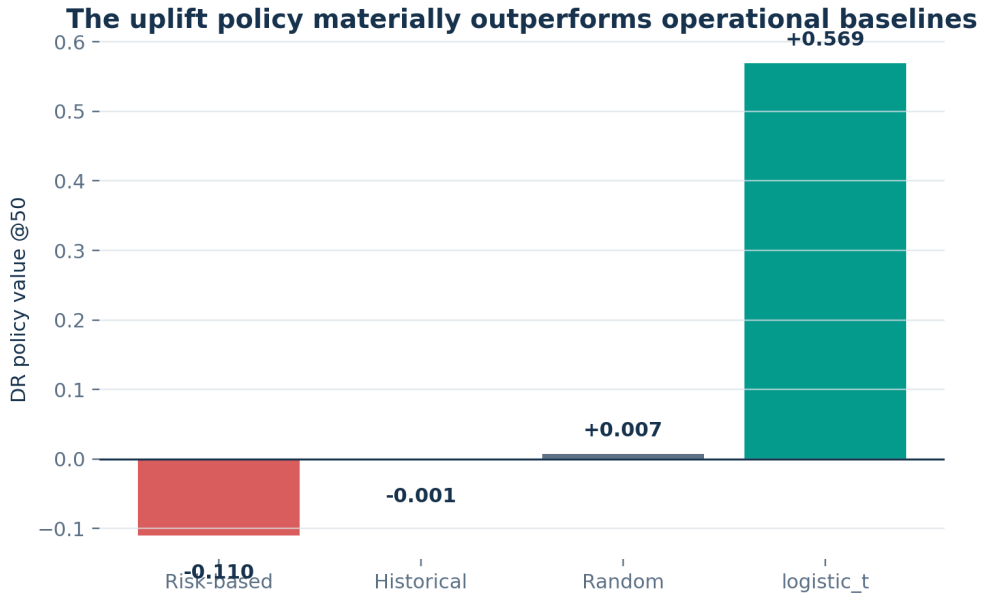


Figure 12: Reported holdout policy values for the champion and operational baselines.
Source: metrics/final_summary.json.

The risk-based ranking is estimated to perform worse than random contact under the current DR evaluation. This is the central business insight: customers with the highest modeled churn risk are not automatically those most likely to be retained through contact.

Magnitude check

The top-50 mean predicted uplift is 0.3578, equivalent to approximately 17.9 model-expected churn events avoided across 50 contacts if probabilities were perfectly causal and calibrated. The reported top-50 DR pseudo-outcome mean is 0.5692, equivalent to 28.5 events. The gap is large and should trigger sensitivity analysis rather than being presented as a definitive operational effect.

8 Calibration, Support, and Subgroup Diagnostics

8.1 Calibration

Potential-outcome probabilities should be calibrated separately because a systematic offset between p_0 and p_1 becomes artificial uplift. The supplied final summary reports ECE of 0.083 for the control model and 0.052 for the treated model.

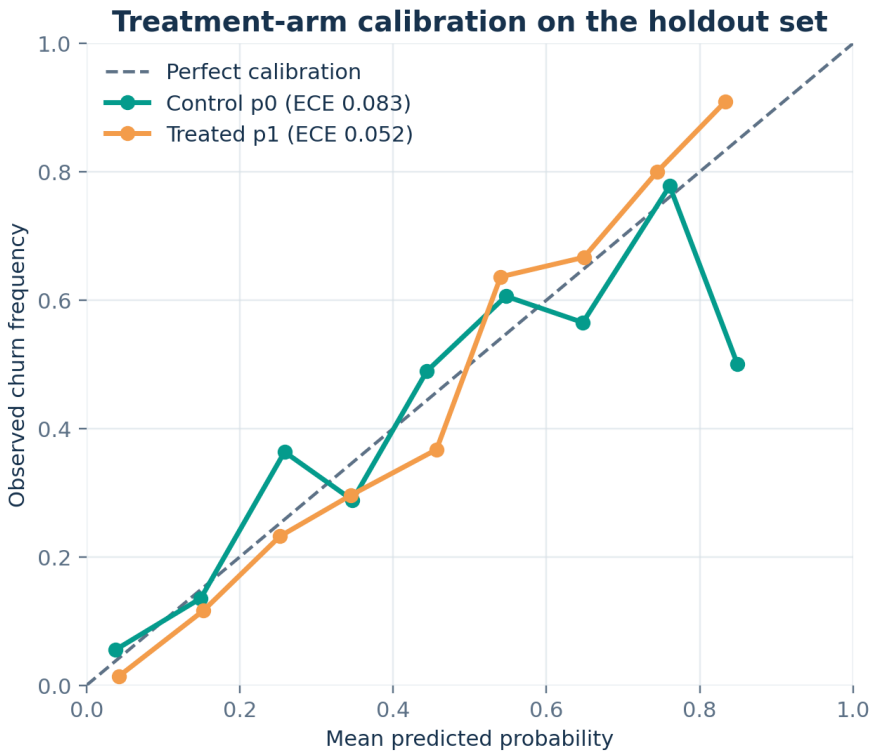


Figure 13: Reliability curves for the two treatment-arm predictions.
Source: Calibration arrays in `metrics/final_summary.json`.

The treated-arm curve is reasonably aligned. The control curve is less stable in upper-probability bins, where sample counts are likely small. The report should include bin counts in a future revision so that deviations can be distinguished from low-support noise.

8.2 Propensity overlap

The holdout propensity range is 0.056 to 0.858, with 1st and 99th percentiles of 0.104 and 0.782. No holdout customers lie outside the configured 0.05–0.95 common-support band.

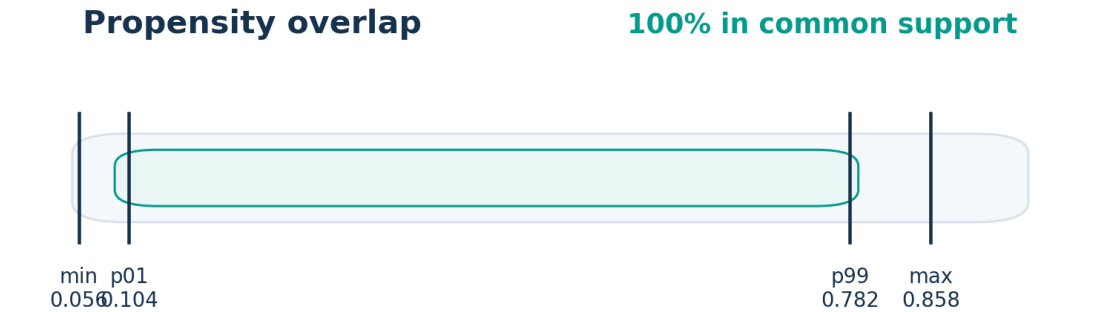


Figure 14: Summary of holdout propensity support.
Source: `metrics/final_summary.json`.

Good measured overlap reduces extreme inverse-propensity weights. It does not demonstrate absence of unmeasured confounding.

8.3 Subgroup performance

Table 5: Holdout subgroup diagnostics

Dimension	Group	N	Control churn	Treated churn	Mean uplift	AUC
subscription_type	Annual	345	35.3%	38.9%	0.090	0.808
subscription_type	Monthly	455	27.3%	27.2%	0.095	0.798
package	Pro	297	27.7%	32.9%	0.092	0.809
package	Basic	398	33.5%	32.2%	0.095	0.791
package	Enterprise	105	26.5%	32.1%	0.088	0.829

All reported segment-level mean uplift values are positive and close to 0.09. However, these are predicted averages, not randomized subgroup treatment effects. The Enterprise subgroup has only 105 holdout customers and requires wider uncertainty reporting.

9 Ranking Output and Business Interpretation

9.1 Top-ranked customers

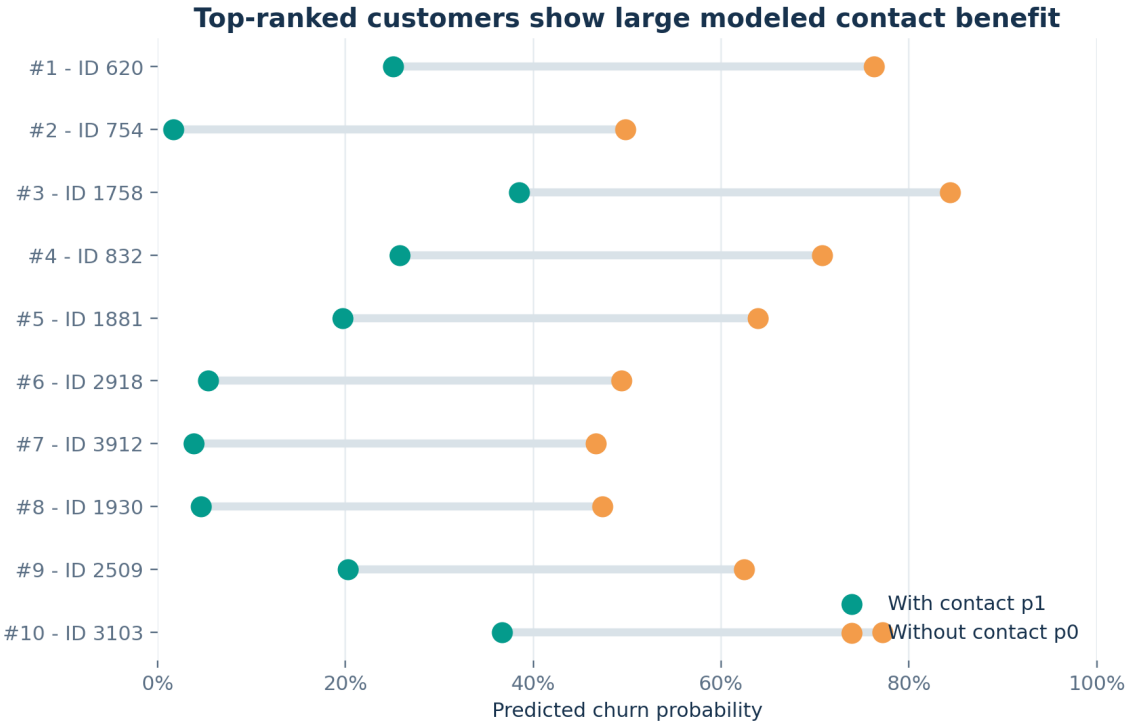


Figure 15: Predicted potential outcomes for the top ten ranked customers.
Source: reports/final_ranking.csv.

Table 6: Top ten ranked customers

Rank	ID	p_0	p_1	Uplift	Propensity	Revenue
1	620	0.762	0.251	0.512	0.685	1,965
2	754	0.498	0.017	0.481	0.698	2,452
3	1758	0.844	0.385	0.459	0.700	5,920
4	832	0.708	0.258	0.450	0.684	774
5	1881	0.639	0.197	0.442	0.535	1,387
6	2918	0.494	0.054	0.440	0.670	3,302
7	3912	0.466	0.038	0.428	0.488	3,773
8	1930	0.473	0.046	0.427	0.564	1,442
9	2509	0.625	0.203	0.422	0.685	439
10	3103	0.772	0.367	0.405	0.685	499

The selected 50 customers have mean predicted no-contact risk of approximately 60.2%, mean predicted contacted risk of approximately 24.4%, and mean modeled uplift of 35.78 percentage points. Their combined monthly revenue is approximately USD 67,135, but the ranking itself is not revenue-weighted.

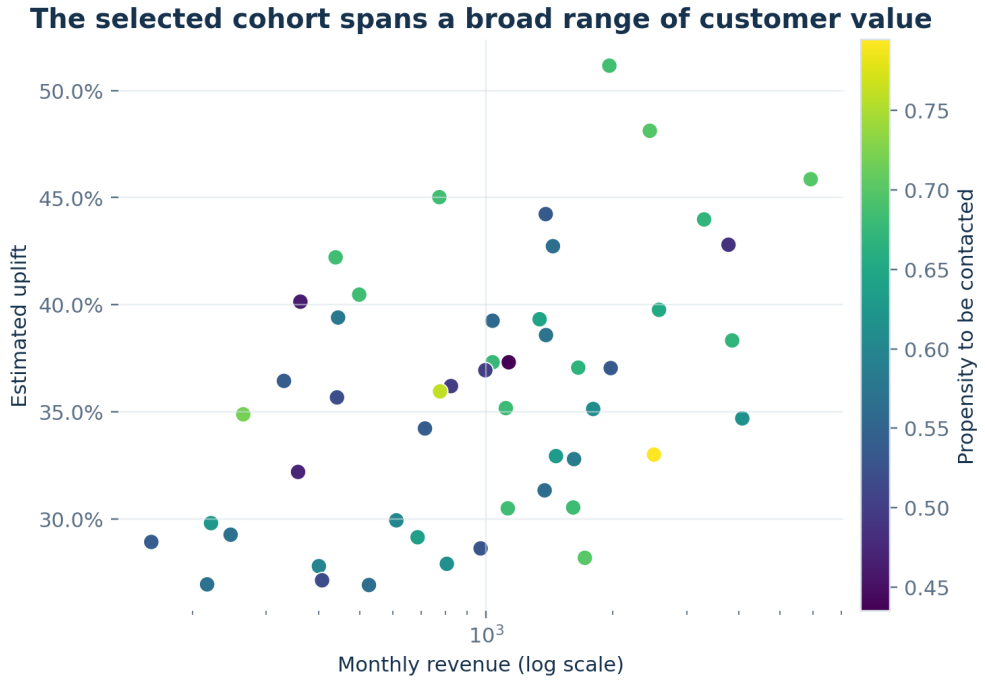


Figure 16: Revenue, estimated uplift, and propensity among the selected cohort.
Source: reports/final_ranking.csv.

9.2 Ranking policy

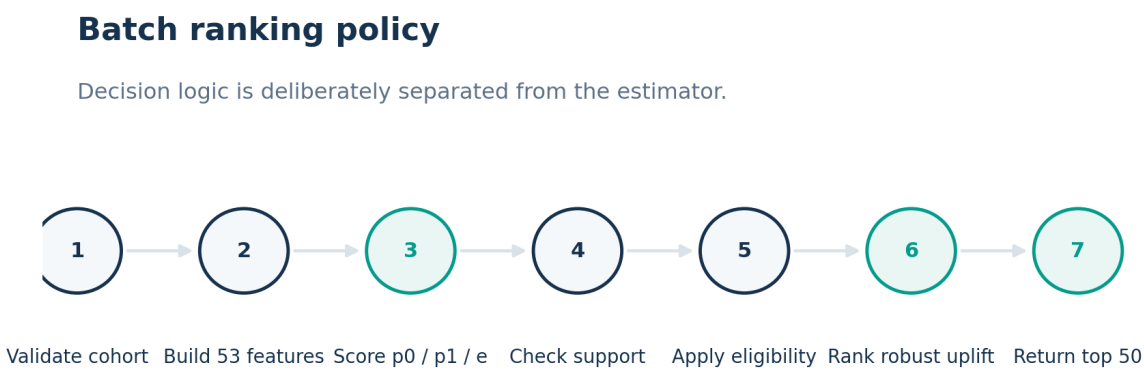


Figure 17: Decision-policy flow used for batch ranking.
Source: Policy modules and `configs/policy.yaml`.

The policy is separated from the estimator and includes eligibility, support, abstention, and deterministic tie-breaking. That separation is a production-strength design choice: the business can change capacity or eligibility without retraining the outcome models.

9.3 Boundary sensitivity

The rank-49 and rank-50 scores differ by only approximately 0.0003. Therefore, the boundary should be treated as operationally fragile. A robust campaign implementation should expose a reserve list and report the selection frequency of customers near the cutoff across repeated splits.

10 Explainability

10.1 Global drivers

The supplied explanation artifacts contain separate importance values for the no-contact outcome model, contacted outcome model, and an uplift-oriented difference. The largest uplift-oriented contributions include email engagement, the first nontrivial usage PCA component, recent zero-day behavior, subscription type, tenure, usage slopes, and relative decline features.

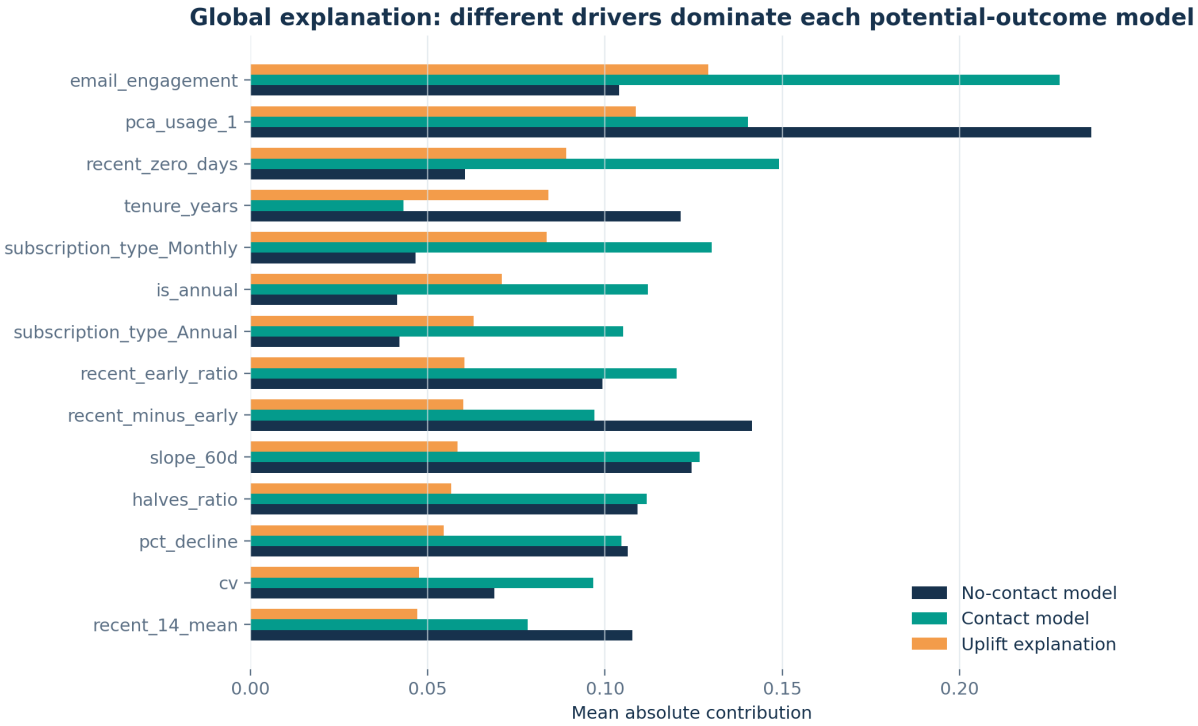


Figure 18: Selected global contribution magnitudes across potential outcomes and uplift explanation.

Source: `global_importance_arm0.json`, `global_importance_arm1.json`, and `uplift_explanation.json`.

10.2 Interpretation requirements

Individual explanations should answer three distinct questions:

1. Why is the customer at risk without contact?
2. Why does the model predict a different risk under contact?
3. Why is the customer ranked above alternatives after support, uncertainty, and policy rules?

SHAP values explain model behavior, not causal truth. Highly correlated usage features should be grouped into business-readable families such as usage level, usage decline, inactivity, support friction, and subscription flexibility.

11 Reproducibility, MLOps, and Deployment Architecture

11.1 Repository and environment

The archived repository contains 104 Python source files and approximately 5661 source lines under `src/`. It uses Python 3.12, `uv`, a committed lockfile, typed configuration, and a console entry point named `retention`.

The last recorded environment contains:

Component	Version
Python	3.12.12
scikit-learn	1.7.2
NumPy	2.3.5
CatBoost	1.2.10
LightGBM	4.6.0
XGBoost	3.1.3
CPU cores	16

11.2 End-to-end architecture

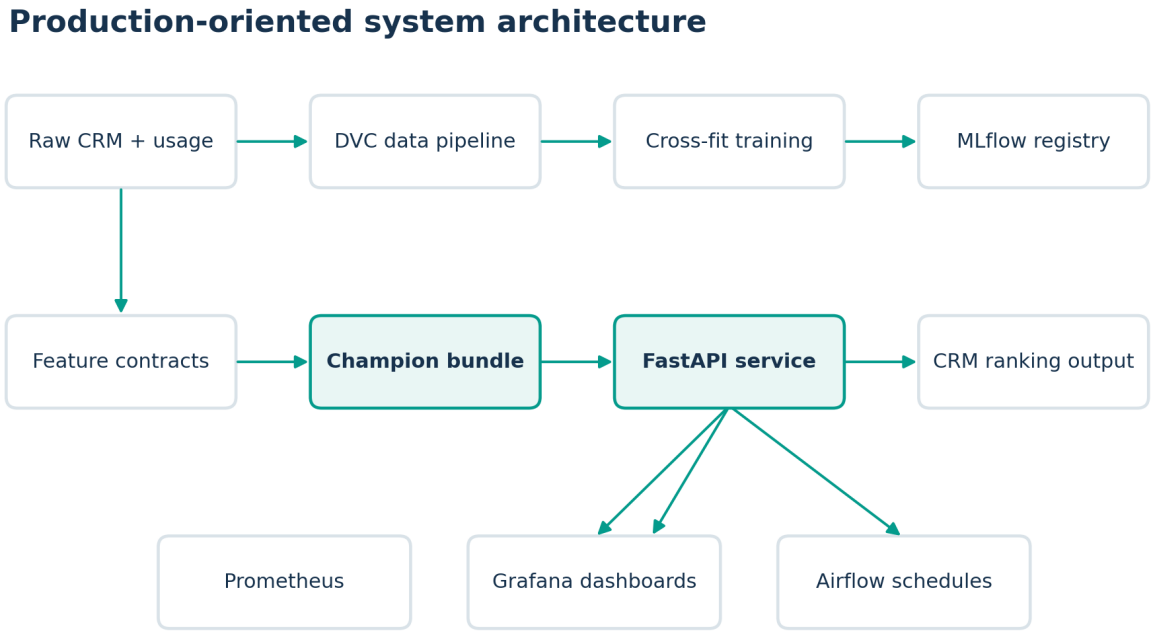


Figure 19: Production-oriented ML system architecture.
Source: DVC, MLflow, bundle, serving, monitoring, and orchestration files in the project snapshot.

11.3 Reproduction commands

```
uv sync --frozen
uv run retention run

# Step-by-step alternative
uv run retention split
uv run retention tune --all
uv run retention train --optuna
uv run retention bundle
uv run retention rank
uv run retention evaluate

# Pipeline and test verification
dvc repro
uv run pytest tests/ -v
```

11.4 MLflow and DVC

The project report records 154 MLflow runs, organized into parent and child runs. The DVC file defines validation, split, training, evaluation, ranking, and bundle stages. The strongest reproducibility requirement is a clean-room execution from raw DVC data through final ranking, followed by test execution and checksum comparison.

11.5 Model bundle

The intended bundle contains the fitted preprocessor, treatment-arm models, calibrators, propensity model, feature manifest, metadata, checksums, model card, and explanation artifacts. The API should call this single bundle so that offline, deserialized, and served predictions remain identical.

12 Serving, Input Contracts, and Failure Engineering

12.1 FastAPI surface

The repository defines liveness, readiness, scoring, ranking, model information, and Prometheus metrics routes. The reported inference latency is approximately 24 ms per customer, with a bundle load time of approximately 6.6 seconds.

Table 7: Serving endpoints

Endpoint	Method	Purpose
`/health`	GET	Process liveness without model dependency
`/ready`	GET	Bundle loaded and ready for traffic
`/v1/score`	POST	Per-customer potential outcomes and uplift
`/v1/rank`	POST	Batch ranking with policy filters
`/v1/model-info`	GET	Immutable model and feature metadata
`/metrics`	GET	Prometheus-compatible aggregate metrics

12.2 Input policy

The codebase includes explicit error codes for duplicated usage days, invalid usage, unknown categories, schema mismatch, negative revenue, engagement bounds, zero company size, model-load failure, inference timeout, ranking failure, and feature-build failure. This is important because production ML quality depends as much on input semantics and fallback behavior as on validation AUC.

12.3 Readiness behavior

Liveness should remain fast even when the bundle is unavailable. Readiness should become true only after artifact loading, checksum and compatibility checks, and a golden smoke prediction. The dependency loader uses caching and a retry path to avoid permanently caching a failed load.

13 Monitoring and Operations

13.1 Three-dashboard model

The project provisions three Grafana dashboards:

1. **Model and business**: uplift distribution, common-support ratio, model status, and global importance.
2. **Operations**: request rate, latency, errors, and active endpoints.
3. **Data and drift**: PSI, coverage, missingness, and schema failures.

13.2 Feature drift

The reported development-versus-holdout PSI values are all below 0.015 for the inspected features, which is very small. This supports the mechanical comparability of the split, although the true production reference should be a time-based historical window rather than a random holdout.

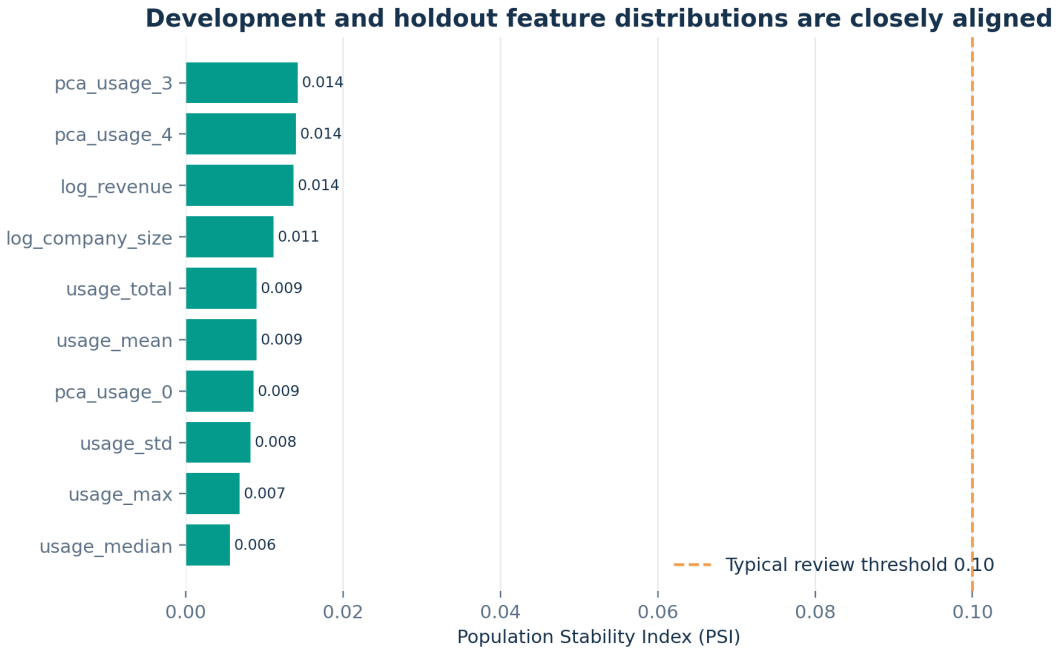


Figure 20: Highest reported PSI values between development and holdout features.
Source: metrics/final_summary.json.

13.3 Prometheus design

The supplied metrics cover HTTP volume and latency, inference counts and duration, estimated-uplift distribution, bundle-load status, schema-validation failures, feature PSI, activity coverage, and negative-uplift counts. Labels are bounded by endpoint, status, feature, and error type. Customer IDs must remain outside Prometheus labels to prevent cardinality explosions and privacy leakage.

13.4 Airflow

The Airflow directory contains scoring and retraining DAG definitions. Airflow is not required for a static interview dataset, but it is appropriate for scheduled source extraction, data-quality gates, recurring ranking, delayed-outcome evaluation, and retraining. Production DAG tasks should call versioned CLI commands or immutable containers rather than duplicating feature logic in the DAG file.

14 Testing and Quality Assurance

The snapshot contains 102 test functions identified through source inspection. The distribution is shown below. The project report states that all tests pass in the original locked environment. In the present review container, full collection was not independently reproduced because optional runtime dependencies such as Pandera, Structlog, and Hypothesis were not installed; this is an environment limitation rather than evidence of a code failure.

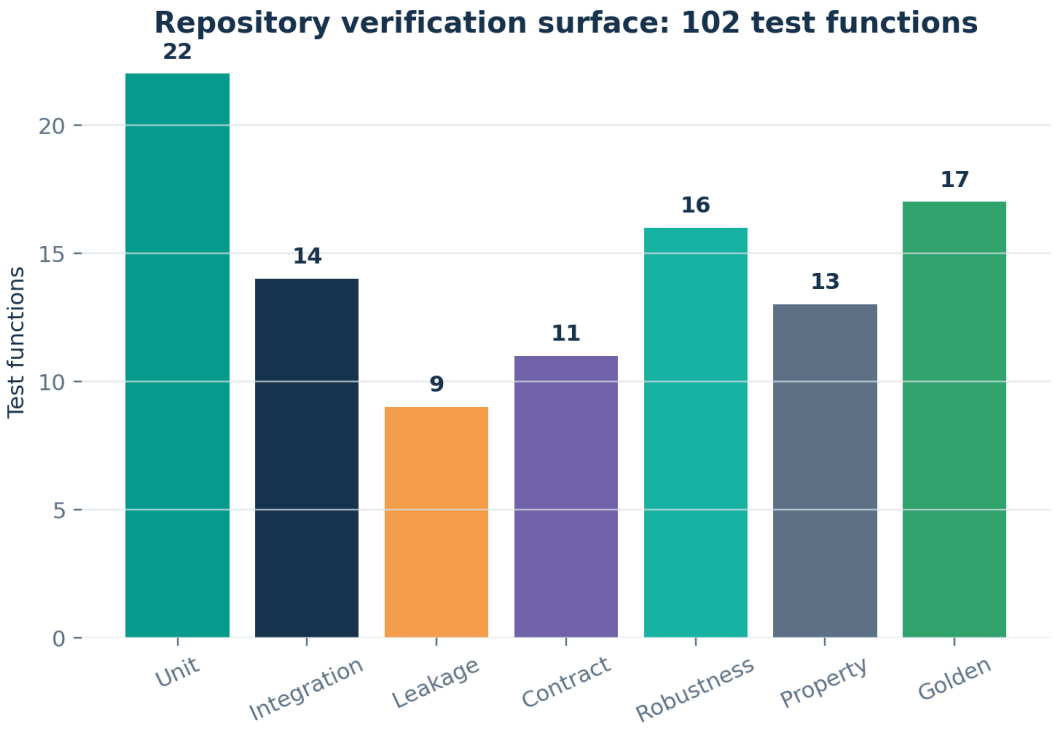


Figure 21: Test-function inventory by category in the supplied snapshot.
Source: Independent AST count of the archived test files.

Table 8: Verification categories

Category	Count	Primary assurance
Unit	22	Feature formulas, calibration, propensity, seeds, uplift sign, validation
Integration	14	API, bundle round-trip, DVC stages, training pipeline
Leakage	9	Holdout isolation, OOF coverage, calibration isolation, temporal cutoff
Contract	11	Input/output schemas and model-feature compatibility
Robustness	16	Batch invariance, column order, extremes, missing days, unseen categories
Property	13	Probability bounds, feature invariants, ranking determinism
Golden	17	Fixtures, known behaviors, end-to-end invariants

The highest-value remaining tests are a true top-50 DR bootstrap test, metadata/champion consistency test, explanation-provenance test, and a clean-container test that regenerates the published ranking hash.

15 Causal Limitations and Experimental Validation

15.1 Unmeasured confounding

The model adjusts only for recorded pre-treatment variables. Historical account-manager judgment, relationship quality, complaint severity, payment status, strategic importance, prior contacts, contact channel, and contact quality may influence both treatment and churn. The reported effects are therefore conditional on an untestable ignorability assumption.

15.2 Treatment definition

A binary `contacted` flag may combine attempts, successful conversations, email, phone calls, and different intervention intensities. A future dataset should distinguish assignment, attempt, reach, channel, timing, duration, message, and outcome.

15.3 Recommended experiment

A production rollout should preserve a randomized control and exploration component. One practical design is:

- 35–40 contacts assigned to the highest conservative uplift scores;
- 5–10 contacts sampled from near-boundary or uncertain customers;
- a matched non-contact control group in the same score bands;
- outcome reporting after a fixed churn horizon;
- intention-to-treat analysis and treatment-compliance reporting.

This creates unbiased data for future policy learning and prevents a self-reinforcing feedback loop.

16 Release-Readiness Review

Required artifact corrections before external release

1. **Champion metadata mismatch.** The model card, README, environment artifact, and comparison file identify `logistic_t`; `metadata.json` identifies a CatBoost T-learner and contains CatBoost parameters. The bundle metadata, model card, API model information, and checksums must be regenerated from one immutable champion run.
2. **Feature-version mismatch.** The final outputs use feature version 2.2.0, while `dataset_manifest.json` retains 2.1.0. The dataset, feature, and model lineage should reference compatible immutable versions.
3. **Bootstrap labeling.** The stored `dr50_bootstrap` is calculated over all holdout DR pseudo-outcomes, not the top-50 selected policy. It is an overall DR mean interval and should not be presented as a confidence interval for DR@50. A valid top-k bootstrap must resample, rerank, and recompute the selected-policy statistic.
4. **Per-customer uncertainty.** Every selected row has `uplift_se=0`. For deterministic logistic models, changing the random seed does not create model uncertainty. Use repeated split refits, customer bootstrap refits, or formal CATE intervals; otherwise omit the SE field.
5. **Stability evidence.** The final summary contains an empty stability block because repeat-level test predictions are not stored in the published NPZ. Persist repeat predictions and report top-50 Jaccard similarity, selection frequency, and rank variance.
6. **Explanation provenance.** Global importance artifacts should be explicitly tied to the logistic champion. SHAP-style CatBoost artifacts must not be served as explanations for a different model family without documented surrogate semantics.
7. **Archive completeness.** The snapshot includes checksums for bundle pickle artifacts but not the binaries themselves. For release, provide the immutable bundle or a DVC/MLflow retrieval reference and verify the checksums in CI.

16.1 Readiness scorecard

Area	Status	Assessment
Business and causal framing	✓ Strong	Correct decision target and explicit causal assumptions
Data integrity	✓ Strong	Complete challenge data and clear validation contracts
Leakage-safe model development	✓ Strong	Dedicated holdout, fold-local preprocessing, repeated cross-fitting
Predictive quality	✓ Strong	AUC and Brier meet stated targets
Policy-value reporting	▲ Review	Strong point estimate, but top-k uncertainty requires correction
Ranking uncertainty	✗ Gap	Zero SE and no stored stability output
Model metadata lineage	✗ Gap	Logistic/CatBoost and feature-version mismatches
Bundle/API architecture	▲ Substantial	Strong implementation surface; binary artifact omitted from snapshot
Testing	▲ Reported	102 test functions present; original locked pass not rerun here
Monitoring and orchestration	✓ Strong design	Three dashboards, alerts, and optional Airflow DAGs
Production causal evidence	✗ Future experiment	Requires randomized validation

17 Recommendations and Next Steps

17.1 Immediate submission actions

1. Regenerate model metadata and model card from the exact `logistic_t` MLflow run.
2. Replace or relabel the bootstrap interval; add a top-50 resample-rerank policy interval.
3. Store repeat-level holdout predictions and produce Jaccard and rank-stability results.
4. Remove the zero per-customer SE field or replace it with a defensible stability measure.
5. Confirm that explanation artifacts correspond to the champion model.
6. Run the full pipeline in the locked Python 3.12 environment and attach a clean-room reproduction log.
7. Generate the one-page challenge summary directly from the corrected machine-readable metrics.

17.2 Modeling roadmap

The logistic T-learner should remain the operational champion until a challenger provides a reproducible improvement in development OOF policy value and stability. Priority challengers are:

- a logistic–CatBoost probability blend with cross-fitted calibration;
- a DR learner with stable logistic nuisance models and a simple Elastic Net CATE stage;
- a compact-feature logistic T-learner to test whether PCA and redundant features are necessary;
- AutoGluon as a nested nuisance-model challenger;
- TabFM only after the conventional evaluation and licensing path is complete.

17.3 Production roadmap

Deploy in shadow mode first, log recommendations and human overrides, and compare them with the existing policy. Move to a randomized limited-capacity campaign, then retrain using treatment assignment, contact compliance, channel, and outcome horizon. Model monitoring should distinguish immediate data/prediction drift from delayed factual and policy metrics.

18 Conclusion

The project demonstrates a mature understanding of the problem: customer retention under limited contact capacity is a causal decision problem rather than a standard churn-classification exercise. The codebase is unusually broad for an interview challenge and incorporates many production concerns that are often omitted: data contracts, chronological feature handling, leakage controls, cross-fitting, multiple causal learners, tuning, experiment tracking, policy separation, serving, monitoring, and tests.

The reported results favor a regularized logistic T-learner, reinforcing an important senior-engineering principle: complexity should be earned by reliable improvement. The project has strong predictive performance and a promising causal ranking, but the estimated effect magnitude remains observational and several artifact-governance inconsistencies should be corrected before the report is treated as a final production release. With those corrections and a randomized validation campaign, the work provides a credible foundation for a real retention decision system.

A One-Page Challenge Submission Summary

Methodology

I modeled the task as uplift estimation rather than conventional churn prediction. For each customer, the system estimates churn risk without contact (p_0) and with contact (p_1), then ranks by $p_0 - p_1$. The selected champion is a regularized logistic T-learner: one model is trained on historical controls and one on contacted customers. Development predictions are generated with five-fold, three-repeat cross-fitting, and the final architecture is evaluated on a stratified 20% holdout.

Use of both data sources. Customer attributes provide company size, package, tenure, contract, revenue, support, and engagement information. The 60-day usage series is transformed into level, recency, trend, inactivity, volatility, shape, missingness, and PCA features. The final manifest contains 53 features.

Evaluation. Factual predictions are evaluated by treatment arm and overall using ROC-AUC, PR-AUC, log loss, Brier score, and calibration. The ranking is evaluated using AUUC, Qini, doubly robust policy value at capacity 50, operational baselines, propensity overlap, subgroup diagnostics, and feature drift.

Result. The reported champion reaches 0.8036 factual ROC-AUC, 0.1626 Brier, and 0.5692 estimated DR policy value at 50. The top 50 have a mean modeled uplift of 0.3578. Risk-only and historical policies are estimated near or below zero under the same DR evaluation.

Ranking-score interpretation. A score of 0.30 means the model estimates a 30 percentage-point lower churn probability under contact than without contact for that customer. This is a conditional observational estimate, not a guaranteed outcome.

Confidence. Confidence is moderate in the predictive ranking structure and lower in the exact causal magnitude. Historical treatment is confounded and individual counterfactual outcomes are never observed. The ranking should be validated through a randomized campaign before full operational adoption.

B Complete Top-50 Ranking

Table 9: Final ranking supplied in `reports/final_ranking.csv`

Rank	Customer ID	p_0	p_1	Uplift	Propensity	Monthly revenue
1	620	0.762	0.251	0.512	0.685	1,964.76
2	754	0.498	0.017	0.481	0.698	2,452.43
3	1758	0.844	0.385	0.459	0.700	5,919.52
4	832	0.708	0.258	0.450	0.684	774.21
5	1881	0.639	0.197	0.442	0.535	1,386.63
6	2918	0.494	0.054	0.440	0.670	3,301.56
7	3912	0.466	0.038	0.428	0.488	3,773.46
8	1930	0.473	0.046	0.427	0.564	1,442.37
9	2509	0.625	0.203	0.422	0.685	438.66
10	3103	0.772	0.367	0.405	0.685	499.11
11	3691	0.710	0.309	0.401	0.462	361.66
12	3608	0.684	0.286	0.398	0.652	2,579.06
13	2967	0.594	0.200	0.394	0.581	444.57
14	1677	0.548	0.155	0.393	0.645	1,340.99
15	1349	0.630	0.237	0.393	0.556	1,036.37
16	2660	0.742	0.356	0.386	0.572	1,388.75
17	2229	0.475	0.092	0.383	0.670	3,849.94
18	1270	0.763	0.390	0.373	0.676	1,036.65
19	1077	0.465	0.092	0.373	0.435	1,132.28
20	2650	0.680	0.310	0.371	0.667	1,656.92
21	3498	0.565	0.195	0.370	0.534	1,979.41
22	3429	0.455	0.086	0.369	0.502	996.09
23	3678	0.472	0.107	0.364	0.539	330.51
24	3018	0.491	0.129	0.362	0.499	825.13
25	2345	0.874	0.514	0.359	0.760	777.39
26	1513	0.524	0.167	0.357	0.520	441.99
27	2020	0.644	0.292	0.352	0.680	1,114.40
28	3177	0.745	0.394	0.351	0.609	1,798.34
29	309	0.742	0.393	0.349	0.719	264.43
30	1054	0.487	0.140	0.347	0.618	4,069.41
31	3371	0.461	0.118	0.342	0.541	715.53
32	530	0.784	0.454	0.330	0.794	2,508.93
33	1863	0.508	0.178	0.329	0.631	1,466.77

Rank	Customer ID	p_0	p_1	Uplift	Propensity	Monthly revenue
34	3805	0.442	0.114	0.328	0.584	1,619.17
35	1558	0.481	0.159	0.322	0.473	357.12
36	2927	0.421	0.107	0.313	0.560	1,378.30
37	2392	0.549	0.244	0.305	0.682	1,611.58
38	3868	0.737	0.432	0.305	0.685	1,126.41
39	2360	0.757	0.458	0.299	0.602	611.80
40	267	0.625	0.327	0.298	0.627	221.65
41	831	0.468	0.175	0.293	0.568	246.78
42	904	0.777	0.485	0.291	0.636	687.39
43	3634	0.566	0.276	0.289	0.540	159.82
44	691	0.587	0.301	0.286	0.530	970.64
45	2852	0.746	0.464	0.282	0.701	1,718.05
46	945	0.553	0.274	0.279	0.614	806.70
47	3236	0.586	0.308	0.278	0.597	400.24
48	3572	0.532	0.260	0.271	0.517	407.28
49	314	0.573	0.303	0.269	0.570	216.97
50	3859	0.361	0.092	0.269	0.563	526.50

C Feature-Drift Detail

Table 10: Highest PSI values in the supplied final summary

Feature	PSI	KS	Shift flag
pca_usage_3	0.0142	0.0203	No
pca_usage_4	0.0140	0.0344	No
log_revenue	0.0137	0.0259	No
log_company_size	0.0111	0.0278	No
usage_mean	0.0090	0.0197	No
usage_total	0.0090	0.0197	No
pca_usage_0	0.0086	0.0200	No
usage_std	0.0082	0.0278	No
usage_max	0.0068	0.0184	No
usage_median	0.0056	0.0175	No

D Source and Artifact Inventory

- Provided challenge brief: `Didar ML Challenge(1).pdf`.
- Project narrative: `report.md`, 988 parsed lines.
- Source snapshot: 341 archived files; 104 source Python files; approximately 5661 source lines.
- Main machine-readable metrics: `metrics/final_summary.json`, `metrics/predictive_metrics.json`, and `metrics/data_quality.json`.
- Ranking: `reports/final_ranking.csv`.
- Model comparison: `reports/model_comparison.csv`.
- Lineage artifacts: `dataset_manifest.json`, `feature_manifest.json`, `metadata.json`, `checksums.json`, and split/seed manifests.
- Operational definitions: `configs/*.yaml`, `dvc.yaml`, `docker-compose.yml`, Prometheus rules, Grafana dashboards, and Airflow DAGs.